



Machine Learning for Petroleum Engineers Using Python

Módulo 5 · Clase 5

¿Cuánta Vida le Queda a la Bomba? — Cierre del Módulo



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: la Última Clase del Módulo

✔ El módulo ya entregó:

- ✔ C2 — una **alarma** que avisa 74 minutos antes de que el choke se tape
- ✔ C3 — una **banda** P10–P50–P90 que cumple lo que promete
- ✔ C4 — una **lista de vigilancia**: a qué campos mirar primero
- ✔ Y el hallazgo que las une: el ML cobra con **muchas unidades y muchas variables**

▶▶ Hoy, la pregunta que falta:

- Detectar (C2) dice «*algo está pasando*». Planear necesita otra cosa: **¿cuánta vida le queda?**
- El equipo que la responde: **gradient boosting**, el modelo que faltaba en la caja de herramientas
- Y el porqué de cada pronóstico, con **valores de Shapley**
- ★ Todo termina en dólares: **¿qué política de mantenimiento es más barata?**

Hoy también se cierra la cuenta del módulo: qué se cubrió del temario y qué se difiere, dicho en voz alta.

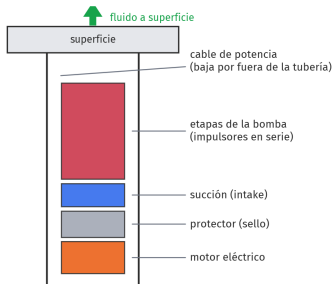
Ustedes Ya Hacen Esto

Nadie llega en cero a esta clase. En sus campos, esto ya existe:

- 🔧 Cuando una ESP lleva dos años corriendo, alguien empieza a decir «*esa bomba ya está pidiendo cambio*». Eso es un pronóstico de **vida útil remanente** — hecho de memoria y con la vida promedio de la flota.
- 📅 El plan de *workover* del año se arma con **run life** promedio por campo. Eso es el *modelo del calendario*: hoy le vamos a poner nombre, número y rival.
- 📈 Y cuando el amperaje del motor empieza a vibrar distinto, el operador *sabe* que algo cambió. Hoy convertimos ese olfato en una tabla que un modelo pueda leer.

La clase no inventa el problema: le pone método al que ya tienen.

La Máquina de Hoy: una Bomba que Trabaja Enterrada



Lo que la va matando

- **desgaste y abrasión (arena)**
los impulsores pierden filo
- **incrustación (escala)**
se angosta el paso del fluido
- **gas libre en la succión**
la bomba cavita y vibra
- **calor en el motor**
el aislamiento envejece

*Nada de esto se ve desde superficie.
Lo que sí se ve: presiones, temperaturas
y velocidades — la huella indirecta.*

Una **ESP** (bomba electrosumergible) es una turbomáquina centrífuga: decenas de impulsores en serie girando a miles de RPM, colgados a 1.500–3.000 metros, dentro del pozo. **No se puede ir a mirar.** Solo mandan señales.

El Día que Se Rompe

Una ESP no se repara: se saca el pozo entero de servicio y se cambia.

Matar el pozo, sacar tubería, correr la bomba nueva, volver a arrancar. Días de equipo de workover y días de producción diferida — si el cambio estaba **programado**.

Si **no** estaba programado, todo empeora: el pozo se queda muerto *esperando* equipo (semanas, si hay cola), la falla puede dañar cable o tubería, y la producción diferida corre todos esos días.

*La misma bomba, el mismo cambio — pero el precio depende de **quién eligió la fecha**: ustedes o la bomba.*

La Cuenta

CAMBIO PROGRAMADO

equipo agendado + 4–6 días
de producción diferida

0,3 MUSD

FALLA EN OPERACIÓN

equipo de emergencia + semanas
de pozo muerto + daños

1,5 MUSD

Son supuestos, no datos — cambian con el campo, el país y el contrato de equipos: en tierra puede ser 5 veces menos; costa afuera, 5 veces más. Lo que suele aguantar es la **proporción**: la falla sorpresa cuesta **del orden de 5 veces** el cambio programado.

Los pone finanzas, no el ingeniero. Los dejamos a la vista para que se puedan discutir y cambiar — toda la clase de hoy cuelga de esos dos números.

Las Tres Políticas Posibles

Con una flota de bombas solo hay tres maneras de decidir **cuándo** cambiar cada una:

-  **Correr a la falla.** No hacer nada hasta que reviente. Se usa toda la vida de la bomba... y *todas* las fallas son de 1,5 MUSD.
-  **Calendario.** Cambiarla a una edad fija, digamos la vida promedio de la flota. Es la política más común del mundo real.
-  **Por condición.** Escuchar los sensores de *cada* bomba y cambiarla cuando *ella* avise. Para eso hay que saber leer el aviso — y eso es lo que un modelo puede aprender.

*Hoy las tres compiten con dinero de verdad sobre la misma flota. No vamos a suponer cuál gana: lo vamos a **medir**.*

El Encargo

Operamos una flota de 100 unidades. El equipo de intervención hay que pedirlo con anticipación y las fallas sorpresa cuestan 5 veces más.

Para cada unidad: ¿cuánta vida le queda? ¿Cuándo pido el equipo? ¿Y cuánto ahorra eso frente al calendario?

Tres entregables, en orden:

1. Un número por unidad: la **vida útil remanente** (RUL), con su versión prudente
2. Una **regla de decisión**: cuándo pedir el equipo, y la lista del mes
3. El **porqué** de cada nombre de la lista — sin porqué, nadie la firma

Acotar el Problema (Regla de la Casa)

- ✓ **Dato real, flota real:** 100 turbomáquinas gemelas del simulador C-MAPSS de NASA, cada una operada **desde sana hasta la falla**, con 21 sensores. Es el dataset de mantenimiento predictivo más estudiado del mundo (PHM 2008).
- ⊘ **Lo que NO es:** una flota de ESP. No existe dato público de fallas de ESP — se verificaron ocho fuentes; el registro RIFTS del consorcio es cerrado. Lo decimos como es.
- ✓ **Por qué vale la sustitución:** una ESP es una turbomáquina — ejes que giran, degradación progresiva, huella en presiones y temperaturas. El **método** es idéntico; los **costos** que le colgamos sí son de ESP.
- ⊘ Hoy **no** dimensionamos bombas ni optimizamos gas lift: la decisión de hoy es *cuándo intervenir*. Lo demás se declara al cierre.

Igual que la C2 dijo «un choke, no una bomba»: la sustitución se declara, no se disimula.

La Flota y el Reloj

Cien máquinas corridas hasta la falla, y el número que las ordena

12 – 40 min

Por Qué Se Degrada una Turbomáquina

La física de la degradación es la misma en una ESP a 2.000 metros y en una turbina de NASA:

- ⚙ Las piezas que giran **se desgastan**: erosión, abrasión, holguras que crecen. La máquina pierde **eficiencia** ciclo a ciclo.
- 🔥 Para entregar el *mismo* trabajo con menos eficiencia, hay que **exigirla más**: sube la temperatura interna, cambia el consumo, se mueven las presiones.
- 🔍 Nada de eso se ve a simple vista — pero **queda escrito en los sensores**. La degradación mecánica deja una *huella termodinámica*.

La apuesta de la clase entera cabe en una frase: **si la huella está en los sensores, un modelo puede aprender a leerla — y a decir cuánto falta.**

El Archivo

Python

```
import pandas as pd

url = ("https://raw.githubusercontent.com/cmosquerat/slb-diplomado/"
      "main/datos/"
      "flota_turbomaquinas_nasa.csv")
d = pd.read_csv(url)

print(d.unidad.nunique(), "unidades")
print(len(d), "filas")
print(d.columns.size, "columnas")
```

>_ Output

```
100 unidades
20631 filas
26 columnas
```

- Cada fila es **un ciclo de operación de una unidad**: arranque, trabajo, parada
- La unidad 1 tiene 192 filas: vivió 192 ciclos y su última fila es **el día que murió**
- En una ESP el «ciclo» sería un día de operación: misma estructura, otro reloj

Las Variables, Una a Una

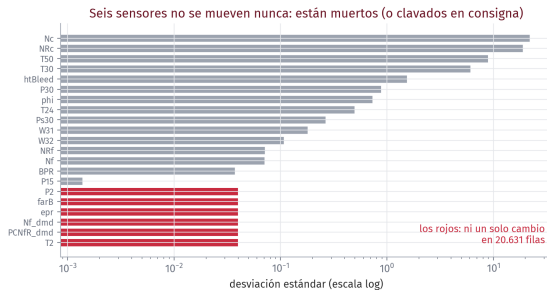
Tres columnas de identidad y régimen, y 21 sensores con su tag industrial. Los que van a importar hoy:

tag	qué mide	en una ESP sería...
T24, T30	temperatura en los compresores	temperatura de motor y etapas
T50	temperatura a la salida de la turbina	temperatura de descarga
P30, P530	presión tras el compresor	presión de descarga
Nf, Nc	velocidad de los dos ejes	RPM del motor
phi	combustible por unidad de presión	consumo eléctrico por barril
BPR	cuánto flujo se desvía del núcleo	reparto de flujo
W31, W32	flujo de refrigeración	enfriamiento del motor

Y las tres de identidad: unidad (cuál máquina), ciclo (su edad), op_1..3 (el régimen de operación — en este subconjunto, constante).

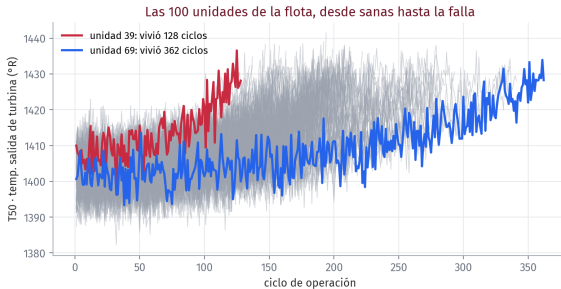
No hace falta ser experto en turbinas: hace falta saber que cada columna es un termómetro o un tacómetro apuntándole a la misma máquina.

Antes de Modelar Nada: Mirar el Dato



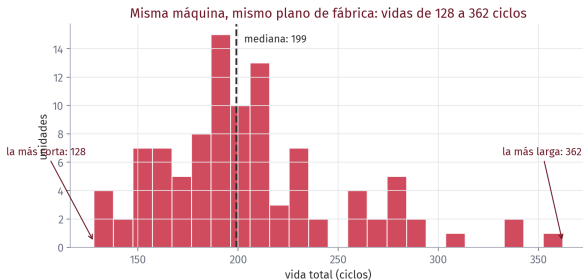
Primer hallazgo del EDA, y nadie lo advierte en la documentación: **6 de los 21 sensores no cambian ni una vez** en 20.631 filas (T2, P2, epr, farB, Nf_dmd, PCNfR_dmd). Al modelo no le aportan nada: afuera.

Cien Vidas, Una Encima de Otra



El sensor T50 de las 100 unidades. Todas arrancan parecido, todas **suben** hacia el final — la huella termodinámica — pero cada una a su ritmo, y cada una muere a otra edad.

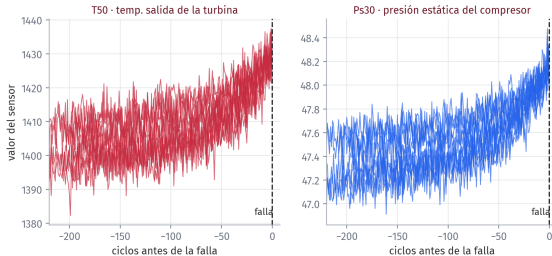
Y Aquí Muere el Calendario Ingenuo



Misma máquina, mismo plano de fábrica: la más corta vivió **128 ciclos** y la más larga **362** — casi 3 a 1. Un calendario a la mediana (199) llega **tarde** a la mitad de la flota y **bota vida** en la otra mitad. Esto no es un defecto del dato: las run life de ESP reales se dispersan igual.

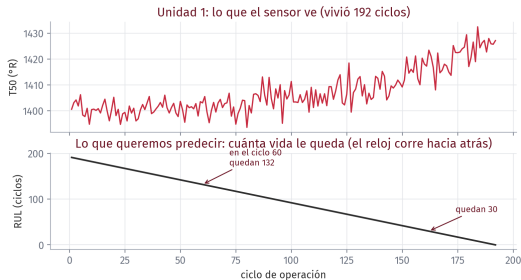
La Huella Común: Alinear por la Muerte

Doce unidades alineadas por su muerte: la degradación tiene una huella común



El truco de la C4 (alinear por un evento, no por el calendario), ahora alineando por la falla: vistas así, doce unidades cuentan **la misma historia** — T50 sube, Ps30 baja. La degradación es legible. Queda convertirla en un número.

El Número: la Vida Útil Remanente (RUL)



RUL (*remaining useful life*): en cada instante, cuántos ciclos le faltan a la unidad para morir. Es un reloj que corre hacia atrás — y como estas 100 unidades ya murieron, **conocemos la respuesta correcta en cada fila**: material perfecto para entrenar y examinar.

RUL, Dicho con Todas Sus Letras

Vale la pena detenerse, porque este vocabulario es el del mantenimiento predictivo en cualquier industria:

término	qué significa, sin adorno
Run life	La vida total del equipo: de la instalación a la falla. En ESP se mide en días; aquí, en ciclos. Es el número que ya manejan.
RUL	<i>Remaining Useful Life</i> : cuánta run life falta desde hoy. $RUL = \text{vida total} - \text{edad}$. Al instalar, $RUL = \text{run life}$; el día de la falla, $RUL = 0$.
Falla funcional	El momento en que el equipo ya no cumple su función (la bomba no levanta caudal), no necesariamente la rotura total. Ahí termina el reloj.
Prognosis (PHM)	La disciplina que estima RUL desde sensores. Es la evolución del <i>monitoreo</i> : detectar dice «está enferma»; prognosis dice «cuánto le queda».
Mant. por condición	La política que decide con el estado medido de <i>cada</i> equipo — lo opuesto al calendario. Necesita la prognosis para planear.

*La física que hace todo esto posible: la degradación mecánica es **gradual y acumulativa** — desgaste, erosión, fatiga avanzan ciclo a ciclo, no de golpe — y cada avance **mueve variables termodinámicas medibles**. Por eso el RUL se puede estimar. Cuando la falla es súbita y sin precursor (un rayo, un golpe), no hay RUL que valga: ningún modelo ve lo que no deja huella.*

El Reloj, en Código

Python

```
vida = (d.groupby("unidad")
        .ciclo.max())

d = d.join(vida.rename("vida"),
          on="unidad")
d["rul"] = d.vida - d.ciclo

d.loc[d.unidad == 1,
      ["ciclo", "vida", "rul"]
      ].iloc[[0, 100, -1]]
```

>_ Output

ciclo	vida	rul
0	1	192
100	101	192
191	192	192

- `groupby().max()` → la vida de cada unidad: su último ciclo
- `vida - ciclo` → el reloj: empieza en la vida y termina en 0 el día de la falla
- Tres líneas — y el problema ya es de **regresión**, como los del M3-C1

De Señales a Tabla, Otra Vez

Regla de la C2, que hoy se recicla entera: **los modelos no comen señales, comen tablas**. Para cada fila, tres miradas por sensor:

- 📷 **El valor de hoy** (T_{50}): la foto instantánea — con su ruido encima
- 📉 **La media de los últimos 20 ciclos** (T_{50_m}): la misma foto, sin temblor — la media móvil del M2, al trabajo
- 📈 **La pendiente de esa media** (T_{50_d}): ¿está subiendo o bajando? — la *dirección* del deterioro

15 sensores vivos $\times$ 3 miradas + la edad de la unidad = **46 columnas**. Muchas unidades, muchas variables: el terreno donde el módulo ya midió que el ML cobra.

*¿Por qué darle la edad? Porque el calendario la usa — y el modelo tiene que poder ganarle **con sus mismas armas y mejores ojos**. Al final vamos a auditar cuánto la usó de verdad.*

La Trampa de Siempre, por Última Vez

Vamos a apartar **25 unidades completas** antes de tocar nada. Entrenamos con 75, y las 25 apartadas solo se destapan al final.

❌ PARTIR POR FILAS

Filas de la *misma* unidad caen en entrenamiento y en examen. El modelo «reconoce» a la unidad — el ciclo 130 de la unidad 5 delata a su ciclo 131 — y el examen sale mentirosamente bien.

✅ PARTIR POR UNIDAD

La unidad entra **entera** a un solo lado. El examen imita la vida real: pronosticar una máquina que **jamás se ha visto**. Es el GroupKFold de la C2 y el «sacar el campo entero» de la C3.

*Tercera clase seguida con la misma trampa. No es casualidad: es **el** error que más informes de ML entierra en esta industria.*

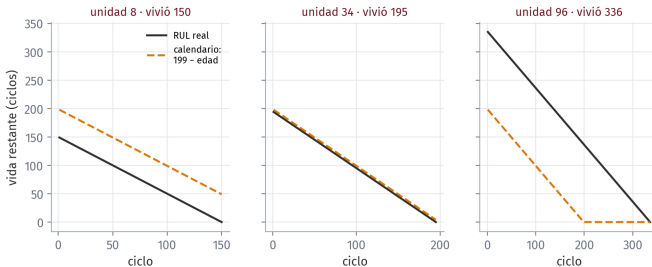
El Modelo que Falta

El calendario como rival, y la cadena que corrige sus propios errores

40 – 70 min

El Rival No Es un Muñeco de Paja: Es la Política Real

El rival: «todas duran lo mismo». Acierta en la del medio — por suerte



El modelo tonto de hoy: $RUL = 199 - \text{edad}$ (la mediana de la flota menos los ciclos corridos). Eso no es un invento académico: **es literalmente el mantenimiento por calendario** que corre en media industria. Ganarle al tonto = justificar el proyecto frente a la política vigente.

El Récord a Batir

Corrimos el calendario sobre las 25 unidades apartadas. Esto es lo que hay que superar:

ERROR, TODA LA VIDA

35 ciclos

de equivocación media
al estimar el RUL

CUANDO QUEDAN < 50

21 ciclos

de error justo cuando
hay que decidir

SI SE LE HACE CASO

14 de 25

unidades revientan antes
de su cita de cambio

*Fíjense en la tercera caja: un error de RUL no es un número abstracto. **Equivocarse por arriba significa una bomba muerta esperando equipo. Volveremos a eso con dólares.***

Qué Falta en la Caja de Herramientas

El diplomado ya les dio: regresión lineal y logística (M3), árboles y **Random Forest** (M3), SVM (M3), K-means (M4). ¿Qué agujero queda?

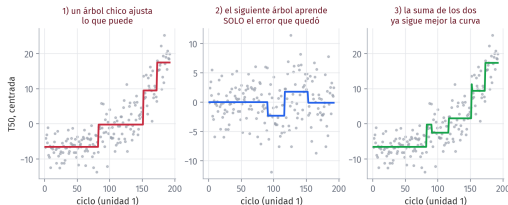
El Random Forest es un **comité**: mil árboles opinan **en paralelo**, cada uno por su lado, y se promedia.

Nadie del comité ve los errores de los demás. ¿Y si en vez de promediar opiniones independientes, cada árbol nuevo **corrigeria lo que el anterior dejó mal**?

Esa idea — árboles **en serie**, cada uno entrenado sobre el error del acumulado — se llama **gradient boosting**. Es el modelo que desde hace una década gana casi todo lo que se juega con **datos tabulares** (filas $\times$ columnas), y el que más corre en producción en la industria.

Hoy lo abrimos despacio, como hicimos con el bosque en el M3: primero la idea con dibujos, después el código, después el examen.

Un Paso de la Cadena, en Cámara Lenta



El segundo árbol **no ve los datos: ve los errores del primero**. Boosting es repetir esa corrección cientos de veces.

La Cadena Completa: 1, 10, 300 Árboles



El RUL de la unidad 34 (apartada: el modelo jamás la vio). Con 1 árbol, una escalera burda; con 10, la forma aparece; con 300, sigue el reloj — incluso el codo donde la degradación se acelera.

Por Qué se Llama «Gradiente», Sin Álgebra

- ⚠ Imaginen el error total del modelo como una **montaña**: la cima es «me equivoco mucho», el valle es «me equivoco poco».
- 👣 Cada árbol nuevo es **un paso cuesta abajo**: apunta en la dirección que más reduce el error *en este momento*. «Gradiente» es solo el nombre técnico de esa dirección.
- ⚙ Y hay una perilla: el **tamaño del paso** (*learning rate*). Pasos grandes bajan rápido pero se pasan de largo; pasos chicos llegan más lejos — si la cadena es larga.



El Estado del Arte, con Fecha y con Límites

Digámoslo con precisión, porque «estado del arte» se dice mucho y se mide poco:

- ✓ En **datos tabulares** — filas y columnas, como los de toda esta industria — el gradient boosting (XGBoost 2016, LightGBM 2017) domina las competencias y la producción **desde hace una década**, y los estudios comparativos serios lo siguen confirmando frente a redes neuronales.
- ✗ En **imagen, texto y señal cruda**, ganan las redes profundas — ahí el boosting no compite.
- ✓ Nuestra tabla de hoy: 20.631 filas $\times$ 46 columnas. **Territorio boosting.**

*Traducción práctica: antes de pensar en una red neuronal para un problema de tabla en su campo, el boosting es **el listón que esa red tendría que superar** — y casi nunca lo supera.*

El Modelo, en Código

Python

```
from sklearn.ensemble import (  
    HistGradientBoostingRegressor)  
  
m = HistGradientBoostingRegressor(  
    max_iter=300,          # arboles  
    learning_rate=0.03,    # paso  
    random_state=0)  
  
m.fit(X_tr, y_tr)          # 75 unid.  
rul_hat = m.predict(X_te)  # 25 unid.
```

- Hist... es el gradient boosting **moderno** de scikit-learn: la misma idea de la cadena, con histogramas para ir rápido (la técnica de LightGBM)
- max_iter → el largo de la cadena;
learning_rate → el tamaño del paso
- La interfaz es **idéntica** a todo lo que ya usaron: fit / predict. Cambiar de modelo cuesta una línea

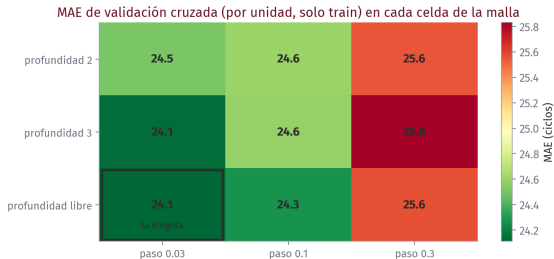
Las Perillas No Se Giran a Ojo

¿Y de dónde salió `learning_rate=0.03`? **No de la intuición:** de un examen. Eso es el *hyperparameter tuning*, y tiene tres reglas que valen más que cualquier receta:

1. **Se prueba una malla, no una corazonada:** todas las combinaciones de paso $\{0,03, 0,1, 0,3\} \times$ profundidad $\{2, 3, \text{libre}\}$
2. Cada celda se califica con **validación cruzada por unidad** (el `GroupKFold` de hace un rato): 5 exámenes por celda, máquinas nunca vistas en cada uno
3. Todo ocurre **dentro de las 75 de entrenamiento**. Las 25 apartadas *no votan*: si la malla las viera, el examen final quedaría contaminado

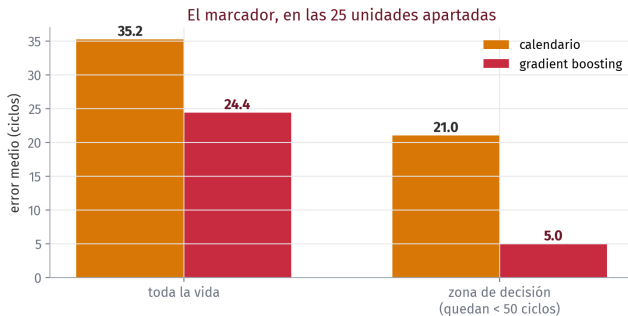
En código son cuatro líneas: `GridSearchCV(modelo, malla, cv=folds_por_unidad)`. Lo difícil no es la sintaxis: es **resistir la tentación de elegir con el test**.

El Resultado de la Malla — y una Lección Incómoda



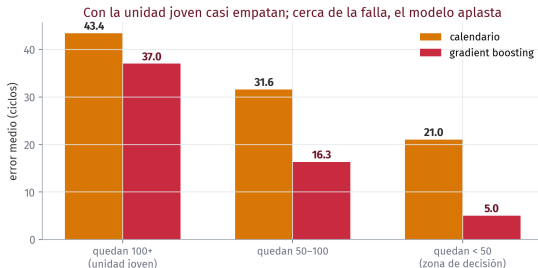
Toda la malla vive entre **24,1 y 25,8 ciclos**; la celda elegida le gana al default de sklearn por **0,2**. La lección honesta: el afinado fino compra *poco* — los saltos grandes los dieron **el modelo correcto y los features correctos**. Afinen al final, no primero.

El Marcador



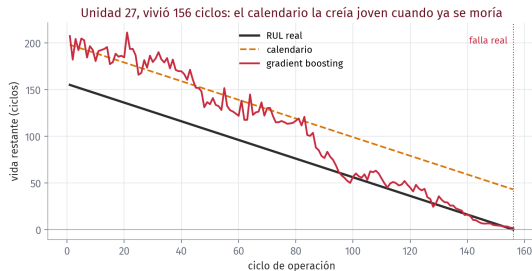
En las 25 unidades nunca vistas: el calendario se equivoca por **35 ciclos**, el boosting por **24**. Gana — pero el marcador global esconde lo importante. Hay que mirar *dónde* se gana.

La Zona de la Decisión



Partido el error por cuánta vida quedaba: con la unidad **joven**, modelo y calendario casi empatan (37 vs 43). Pero cuando quedan menos de 50 ciclos — **justo cuando se decide pedir el equipo** — el calendario yerra por 21 y el modelo por 5. La plata vive ahí.

Una Unidad Vista de Cerca



La unidad 27 vivió **156 ciclos** — de las cortas. El calendario (199 — edad) la creía con 43 ciclos de vida **el día que murió**. El modelo la ve degradarse y **dobra hacia cero con ella**. Esta lámina es la clase entera en una figura.

El Resultado Negativo de Hoy: la Unidad Joven

Mientras la máquina está sana, el modelo NO sabe más que el calendario.

Con 100+ ciclos por delante los sensores todavía no muestran degradación — y sin huella que leer, no hay nada que aprender: 37 vs 43 ciclos de error, casi empate.

¿Por qué esto es una *buena* noticia? Porque delata cómo funciona de verdad: **el modelo no adivina el futuro — reconoce la degradación presente y la proyecta.** Cuando no hay degradación, no promete nada.

Vacuna para la vida profesional: si un proveedor les ofrece predecir fallas «con seis meses de anticipación» en equipos sin degradación observable... ya saben qué preguntarle.

El Porqué y el Precio

Shapley para explicar cada pronóstico, y dólares para elegir la política

70 – 95 min

Un Pronóstico Sin Porqué No Se Firma

El modelo dice: «a la unidad 23 le quedan 43 ciclos». El supervisor que tiene que pedir el equipo va a hacer **una sola pregunta**: ¿por qué? Y hay dos maneras de responderla:

🌐 GLOBAL: ¿en qué se fija el modelo?

En general, en toda la flota. Ya la conocen: es la **importancia por permutación** de la C3 — desordenar una columna y medir cuánto duele.

🔍 LOCAL: ¿por qué ESTE número, HOY?

Para esta unidad, en esta foto. Es la pregunta de la orden de trabajo — y la nueva de hoy. La responde un señor llamado **Shapley**.

*La global sirve para confiar en el modelo. La local sirve para firmar **una** decisión. Hacen falta las dos.*

Shapley: Repartir el Crédito de una Jugada

Lloyd Shapley (Nobel de economía 2012) resolvió un problema de juegos en equipo que es *exactamente* el nuestro:

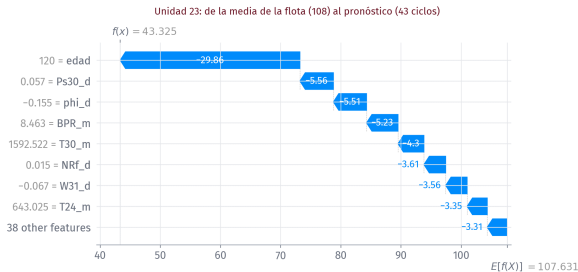
Un equipo ganó un premio. ¿Cómo repartirlo **justo** entre los jugadores? Respuesta: midiendo cuánto cambia el resultado **con y sin cada jugador**, en todas las alineaciones posibles, y promediando.

Traducido a hoy:

- El «premio» es la **distancia** entre el pronóstico promedio de la flota y el pronóstico de esta unidad
- Los «jugadores» son las **46 columnas**: cada una recibe su pedazo del empuje, en ciclos, con signo
- Para modelos de árboles esto se calcula **exacto y rápido** (TreeSHAP) — por eso hoy es el estándar de la industria

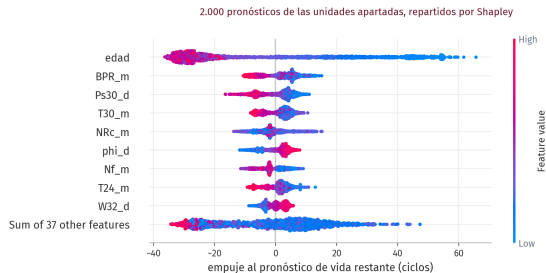
En código: `shap.TreeExplainer(modelo)`. Dos líneas. Lo difícil — como siempre — es leerlo bien.

El Porqué de la Unidad 23, Pieza por Pieza



Se lee de abajo hacia arriba: la flota promedia **108** ciclos de RUL; la edad de esta unidad empuja **-30**; la presión del compresor cayendo (Ps30_d), el consumo torciéndose (phi_d) y el resto suman su parte... pronóstico final: **43**. El RUL real era **48**. Esto ya es un párrafo de orden de trabajo.

La Flota Entera, Repartida por Shapley



Cada punto es un pronóstico; el color, el valor del sensor. Léanlo como ingenieros: BPR alto (rojo) empuja la vida **hacia abajo**, Ps30 cayendo también. El modelo redescubrió la termodinámica de la degradación — **sin que nadie se la contara**.

La Letra Chica de Shapley (Léanla Siempre)

Shapley reparte el pronóstico DEL MODELO. No reparte la culpa física.

Si dos sensores cuentan la misma historia (y en una máquina degradándose, casi todos la cuentan), se **roban crédito** entre sí. El reparto es fiel al modelo, no al mecanismo.

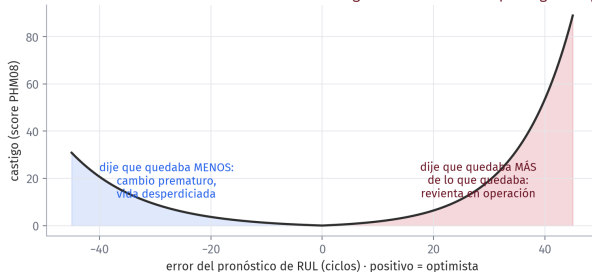
La prueba, con nuestros propios números:

- En el waterfall, la **edad** se llevó –30 ciclos: el pedazo más grande por lejos
- Pero cuando **le quitamos la edad** al modelo y lo reentrenamos... el error casi no cambió: 24,2 contra 24,4
- Conclusión: la información de la edad **también vive en los sensores** (una máquina vieja se ve vieja). Shapley le dio el crédito a la edad porque el modelo la usó primero — no porque fuera imprescindible

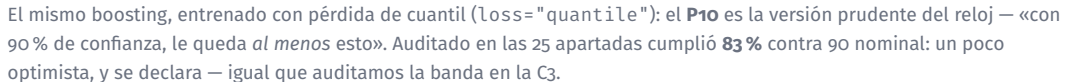
*Ese experimento se llama **ablación**: quitar una pieza y medir. Shapley les dice qué usó el modelo; la ablación, qué necesitaba. Con las dos, la explicación aguanta una reunión hostil.*

El Error Tiene Dirección, y los Precios También

Hasta el benchmark de NASA califica asimétrico: llegar tarde cuesta más que llegar temprano



Decir «le quedan 60» cuando quedaban 20 → bomba muerta esperando equipo: **1,5 MUSD**. Decir «le quedan 20» cuando quedaban 60 → vida desperdiciada: mucho más barato. Hasta el examen oficial del PHM08 castiga así, con exponenciales asimétricas. **El P50 ignora esto. La decisión no puede.**



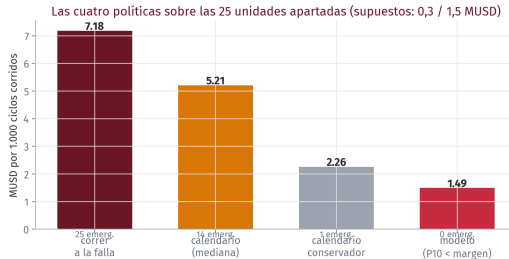
La Regla de Decisión, Completa en Tres Líneas

1. Cada ciclo, mirar el **P10** del RUL de cada unidad.
2. Si baja de **20 ciclos** → pedir el equipo hoy.
3. El cambio ocurre **15 ciclos después** (lo que tarda la movilización).

Cada número tiene dueño: el **15** es de logística (cuánto tarda el equipo en llegar — pregúntenle a operaciones, no al modelo). El **20** es el 15 más un colchón que paga la imperfección del P10. Y el **P10** — no el P50 — es porque el error hacia arriba cuesta 5 veces más.

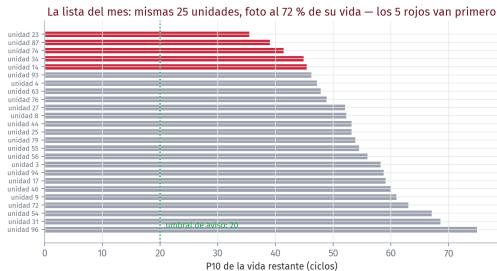
Noten qué pasó: la estadística puso el pronóstico, la logística puso el 15, y finanzas puso el P10. Una regla de decisión de verdad siempre tiene esas tres firmas.

Las Cuatro Políticas, en Dólares



Sobre las 25 unidades apartadas, vida completa, mismos supuestos: correr a la falla cuesta **7,18**; el calendario común, 5,21 (¡14 emergencias!); el calendario conservador se salva de las emergencias pero **bota 41 ciclos** de vida por unidad: 2,26. El modelo: **0 emergencias, 5 ciclos botados, 1,49** — un **34 % más barato** que el mejor calendario posible.

El Entregable: la Lista del Mes



La foto de hoy de la flota, ordenada por el P10. Los cinco rojos van primero — y gracias a Shapley, **cada uno lleva su porqué** («Ps30 cayendo, consumo subiendo») en la orden de trabajo. Esto es lo que se firma. El resto de la flota: a seguir corriendo.

Cómo Reportarlo al que Decide

❌ ASÍ NO

«Entrenamos un HistGradientBoostingRegressor con malla de hiperparámetros por GroupKFold y logramos un MAE de 24,4 contra 35,2 del baseline.»

Todo cierto. Nada decidable.

✅ ASÍ SÍ

«Con los sensores que ya tenemos, el plan de cambios pasa de 14 fallas sorpresa a **cero**, botando 5 ciclos de vida por bomba en vez de 41. Es un **34 % menos** de costo de mantenimiento por ciclo corrido que el mejor calendario.

La lista de este mes tiene 5 bombas, cada una con su motivo.»

La versión de la derecha se puede llevar a un comité de presupuesto. Y cada número de ese párrafo salió de una lámina de hoy — ninguno es de fe.

Su Turno

Otro costo, otra decisión — el mismo modelo

95 – 118 min

Su Turno: Firmen Su Propia Regla

🕒 20 min

En el cuaderno está todo montado: el modelo entrenado, el P10 por unidad, y la simulación de políticas **como función**. Ustedes operan un campo distinto al de la clase:

EL ESCENARIO B — CAMPO EN TIERRA

Cambio programado: **0,08 MUSD** · Falla en operación: **0,16 MUSD** (hay equipo cerca: la sorpresa solo cuesta el doble, no cinco veces). La movilización tarda **5** ciclos.

1. Corran las cuatro políticas con *sus* costos. ¿Sigue ganando el modelo? ¿Por cuánto?
2. Ajusten el umbral de aviso (prueben 10, 20, 30) y quédense con el más barato. ¿Les dio distinto que a la clase? ¿Por qué?
3. Escriban el párrafo «Así Sí» para *su* gerente, con sus números

Pista de la trampa: cuando la sorpresa cuesta casi lo mismo que el plan, la prudencia deja de ser gratis. No busquen la respuesta de la clase: busquen la de su campo.

Lo Que Deben Encontrar (No Lean Antes de Intentar)

El punto andragógico del ejercicio, dicho después de sudarlo:

Mismo modelo, mismos pronósticos, otra estructura de costos → OTRA política óptima.

Con la falla a solo $2\times$, esperar más y tolerar alguna emergencia puede salir más barato que ser prudente. El umbral óptimo se corre, y hasta el calendario puede volver a ser competitivo.

*La decisión nunca estuvo dentro del modelo: estuvo en los costos. El modelo pone el pronóstico; el negocio pone el umbral. Quien entiende eso no vuelve a pedir «el mejor modelo» — pide **el mejor sistema de decisión**.*

Cierre

Lo Que Aprendimos Hoy

lo que vimos	lo que decidimos con eso
Vidas de 128 a 362 ciclos en máquinas gemelas	El calendario ingenuo pierde: 14 sorpresas de 25
La degradación deja huella en los sensores	Convertirla en RUL: el reloj que corre hacia atrás
Gradient boosting: la cadena que corrige	Error de 5 ciclos en la zona de decisión (4× mejor)
La malla afinó con método (GroupKFold)	...y compró 0,2 ciclos: afinar al final, no primero
Shapley reparte el porqué de cada pronóstico	Órdenes de trabajo con motivo — y con letra chica
El P10 y la regla de los tres números	O emergencias y 34 % menos costo que el mejor calendario

*Y el arco del módulo quedó cerrado: la C2 detecta, la C3 acota, la C4 prioriza, la C5 **decide**.*

Los Resultados Negativos de Hoy

Como todas las clases de este módulo: lo que **no** funcionó, dicho en voz alta.

- ✗ **Con la unidad joven, el modelo no sabe más que el calendario** (37 vs 43). Sin degradación visible no hay pronóstico fino — el modelo proyecta el presente, no adivina el futuro.
- ✗ **El afinado compró 0,2 ciclos.** La malla entera vive en 1,7 ciclos de rango. Si esperaban magia de los hiperparámetros: no la hay — la ganancia vivía en el modelo y los features.
- ✗ **El P10 prometió 90 % y cumplió 83 %.** Igual que la banda de la C3 (76 contra 80): los cuantiles aprendidos tienden al optimismo, y por eso el umbral de aviso lleva colchón.

Tres limitaciones medidas y declaradas valen más que una demo perfecta. Esa es la diferencia entre una clase y un folleto.

Si Se Llevan Una Sola Cosa

Un modelo no vale por su error promedio: vale por las decisiones que cambia.

El boosting le ganó al calendario por 11 ciclos de MAE — y eso no convence a nadie. Le ganó por **14 emergencias evitadas y un 34 % del presupuesto de mantenimiento** — y eso aprueba proyectos. La traducción del primero al segundo la hicieron el P10, un umbral con dueño y dos costos puestos a la vista. Esa traducción es el trabajo de ustedes.

Y su hermana chica, para el día a día: cuando les presenten un modelo, pregunten primero «¿contra qué política vigente compite, y en qué unidades se mide la ganancia?». Si no hay respuesta, no hay proyecto.

La Cuenta del Módulo, Como Se Prometió

✓ TEMARIO CONTRATADO: CUBIERTO EN ESTAS CINCO CLASES

DCA probabilístico (C3, con Arps pagado) · **GOR y WOR** (C4) · **Breakthrough de agua** (C4) · **Fallas de equipo de levantamiento** (C2: el método sobre un choke · C5: RUL y decisión, sobre turbomáquinas de NASA — la sustitución de dato, declarada ambas veces) · **Optimización de levantamiento artificial** (C5: cuándo intervenir cada unidad bajo costos — dimensionamiento y gas lift quedan fuera)

Las 3 prácticas comprometidas: entregadas (series de tiempo C2 · WOR C4 · integración volumétrica + forecasting C3).

➡ DIFERIDO AL PROYECTO INTEGRADOR, EXPLÍCITAMENTE

Feature selection para **EOR** · **presión promedio** de yacimiento · análisis de **pruebas de presión** (DST, RFT, buildup) · **daño de formación**. En 10 horas, a la profundidad de este curso, no entraban — y preferimos deberlas a fingirlas. Se retoman sobre los datos de ustedes.

5 de 9 temas a fondo, 3 de 3 prácticas, y las deudas por escrito. Esa es la cuenta.

Datos y Referencias

-  **NASA C-MAPSS** — *Turbofan Engine Degradation Simulation Data Set*, Prognostics Center of Excellence, NASA Ames. Dominio público. Saxena, Goebel, Simon & Eklund (2008), *Damage Propagation Modeling...*, PHMo8.
-  **Lundberg & Lee (2017)** — *A Unified Approach to Interpreting Model Predictions* (SHAP), NeurIPS. Y **Grinsztajn et al. (2022)** — *Why do tree-based models still outperform deep learning on tabular data?*, NeurIPS.
-  El CSV curado se construye con `modulo5-clase5/preparar_datos.py`; las figuras y **todas** las cifras de estas láminas salen de `figuras.py`. Nada está escrito a mano.

Como siempre: clonan el repositorio, corren los dos scripts, y tiene que dar exactamente lo mismo. Si no da, es un error nuestro y queremos saberlo.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 5 · Clase 5 · SLB Ecuador · UDLA · 2026